FILE HANDLING

CAVEAT: bash does not handle binary data properly in versions < `4.4`.

Read a file to a string

Alternative to the `cat` command.

```
file_data="$(<"file")"
```

Read a file to an array (*by line*)

Alternative to the `cat` command.

```
# Bash <4 (discarding empty lines).
IFS=$'\n' read -d "" -ra file_data < "file"

# Bash <4 (preserving empty lines).
while read -r line; do
    file_data+=("$line")
done < "file"

# Bash 4+
mapfile -t file_data < "file"
```

Get the first N lines of a file

Alternative to the `head` command.

CAVEAT: Requires bash 4+

Example Function:

```
head() {
    # Usage: head "n" "file"
    mapfile -tn "$1" line < "$2"
    printf '%s\n' "${line[@]}"
}
```

Example Usage:

```
$ head 2 ~/.bashrc
# Prompt
PS1='→ '

$ head 1 ~/.bashrc
# Prompt
```

Get the last N lines of a file

Alternative to the `tail` command.

CAVEAT: Requires bash 4+

Example Function:

```
tail() {
    # Usage: tail "n" "file"
    mapfile -tn 0 line < "$2"
    printf '%s\n' "${line[@]: -$1}"
}
```

Example Usage:

```
$ tail 2 ~/.bashrc
# Enable tmux.
# [[ -z "$TMUX"  ]] && exec tmux

$ tail 1 ~/.bashrc
# [[ -z "$TMUX"  ]] && exec tmux
```

Get the number of lines in a file

Alternative to `wc -l`.

Example Function (bash 4):

```
lines() {
    # Usage: lines "file"
    mapfile -tn 0 lines < "$1"
    printf '%s\n' "${#lines[@]}"
}
```

Example Function (bash 3):

This method uses less memory than the `mapfile` method and works in bash 3 but it is slower for bigger files.

```
lines_loop() {
    # Usage: lines_loop "file"
    count=0
    while IFS= read -r _; do
        ((count++))
    done < "$1"
    printf '%s\n' "$count"
}
```

Example Usage:

```
$ lines ~/.bashrc
48

$ lines_loop ~/.bashrc
48
```

Count files or directories in directory

This works by passing the output of the glob to the function and then counting the number of arguments.

Example Function:

```
count() {
    # Usage: count /path/to/dir/*
    #        count /path/to/dir/*/
    printf '%s\n' "$#"
}
```

Example Usage:

```
# Count all files in dir.
$ count ~/Downloads/*
232

# Count all dirs in dir.
$ count ~/Downloads/*/
45

# Count all jpg files in dir.
$ count ~/Pictures/*.jpg
64
```

Create an empty file

Alternative to touch.

```
# Shortest.
>file

# Longer alternatives:
:>file
echo -n >file
printf '' >file
```

Extract lines between two markers

Example Function:

```
extract() {
    # Usage: extract file "opening marker" "closing marker"
    while IFS=$'\n' read -r line; do
        [[ $extract && $line != "$3" ]] &&
            printf '%s\n' "$line"

        [[ $line == "$2" ]] && extract=1
        [[ $line == "$3" ]] && extract=
```

```
    done < "$1"
}
```

Example Usage:

```
# Extract code blocks from MarkDown file.
$ extract ~/projects/pure-bash/README.md '```sh' '````'
# Output here...
```